Star Trek — Technical Report

LunarLander-v3, DQN and Double DQN

Team Star Trek — G-AIA-401 Reinforcement Learning

May 2026

This report describes our code and experiments. The main files are `train_dqn.py`, `train_double_dqn.py`, `eval.py`, the YAML files in `configs/`, and the script `run_all.sh`. We train each agent for 2000 episodes, then we evaluate it without exploration (greedy policy). The numbers in section 7 come from the CSV files in `stats/csv/` on this machine.

1 1. Context and goal

We use Gymnasium `LunarLander-v3`. The lander must land between two flags. At each step the agent sees eight numbers: position x and y, speed vx and vy, angle, angular speed, and two leg contacts. It chooses one of four actions: do nothing, left engine, main engine, or right engine.

The reward mixes several parts: distance to the pad, speed and tilt, +10 when a leg touches the ground, fuel cost when an engine fires, +100 for a good stop (Box2D sleep), and -100 for a crash or if the lander leaves the screen.

An episode can end in four ways. Crash on the ground: `terminated = True`. If `abs(x) > 1` the lander left the screen: still `terminated = True`. If Box2D marks the body as asleep on the ground, that is a good landing: `terminated = True`. If the episode hits 1000 steps, Gymnasium sets `truncated = True`. We log both flags in the CSV as the assignment asked.

The assignment also asks for a mean score of at least 200 over 100 evaluation episodes in a row, some measure of spread, and a simple way to rerun everything. We use DQN and Double DQN (value-based methods) and we keep parameters in YAML files.

2 2. Repository layout

Shared settings are in `configs/default.yaml`. Each run adds a second file (`train_dqn.yaml`, `train_double_dqn.yaml`, or `eval.yaml`) so only the differences are listed. Training code is in `src/model_train`, the hand-made baseline is in `src/bad_model`, evaluation is in `src/eval.py`, and small helpers are in `utils/`.

When you run the project, CSV files and plots go to `stats/`, and model weights go to `models/`. The script `run_all.sh` trains DQN, then Double DQN, then runs evaluation for both.

Merged settings used by the scripts:

Parameter	DQN	Double DQN
`lr` (Adam)	5e-4	5e-4
`gamma`	0.99	0.99
`batch_size`	128	128
`buffer_size`	100,000	100,000
`epsilon_start`	1.0	1.0
`epsilon_min`	0.05	0.05
`epsilon_decay`	0.995 / ep.	0.995 / ep.
`warmup_steps`	5,000	5,000
`target_update_every`	8 episodes	5 episodes
`num_episodes`	2,000	2,000
`seed`	42	42

File `utils/seed.py` fixes randomness for Python `random`, NumPy, PyTorch (CPU and GPU), and cuDNN. The same seed is passed to the environment and to exploration. If the policy is deterministic, a second run gives the same numbers. For a trained network, small differences can still appear because of multithreading in PyTorch.

3 3. Baselines

We use two simple policies to compare with the neural network. First, a random policy that picks a legal action with equal chance. If the trained agent is not better than random, it did not learn much.

Second, a short rule-based policy in `src/bad_model/basic.py`:

```
    x, y, vx, vy, angle, angular_vel, left, right = obs

    if left and right:
        return 0
    if angle > 0.15:
        return 1
    if angle < -0.15:
        return 3
    if vy < -0.8:
        return 2
    if vx > 0.5:
        return 1
    if vx < -0.5:
        return 3
    if y < 0.5 and vy < -0.3:
        return 2
    return 0
```

The rules are simple on purpose. For example, firing the main engine when the vertical speed is already very negative often happens too late and wastes fuel. In practice this heuristic can score worse than random because bad episodes pull the mean down, even if it sometimes lands well.

The tables in this report use only `eval_dqn.csv` and `eval_double_dqn.csv` (100 greedy episodes each, see section 7). File `eval_results.csv` may exist from an old default path in the config; we did not use it for the main results. We also did not add a full 5-seed CSV pack for random and heuristic here. You can run `eval.py` five times with seeds 0 to 4 and merge the files if you need that later.

4 4. DQN agent

The Q-network is a small feed-forward network: 8 inputs, two hidden layers of 128 units with ReLU, then 4 outputs (one Q-value per action). We did not normalise the observations (we say more about that in the ablation section).

```
class QNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(8, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 4)
        )
```

We store transitions in a replay buffer (deque, size 100000). We use batches of 128. We wait for 5000 environment steps before the first gradient step (`warmup_steps`). The target network is a full copy of the online network, refreshed every `target_update_every` episodes (8 for DQN, 5 for Double DQN in our YAML). Exploration is epsilon-greedy: epsilon starts at 1.0, is multiplied by 0.995 after each episode, and cannot go below 0.05. With this schedule, epsilon reaches its minimum around episode 600. Optimiser: Adam with learning rate 5e-4. Loss: SmoothL1 (Huber). Gradient clipping with global norm 1.0.

Double DQN changes only the backup: we take the best action with the online network, but we read its Q-value from the target network.

```
    q = model(obs).gather(1, actions.unsqueeze(1)).squeeze()
    next_actions = model(next_obs).argmax(1).unsqueeze(1)
    next_q = target_model(next_obs).gather(1, next_actions).squeeze()
```

5 5. Evaluation

Script `eval.py` loads a checkpoint, runs `num_episodes` episodes, and writes one CSV line per episode with columns `episode`, `reward`, `terminated`, `truncated`. Each reset uses seed `base_seed + episode_index`.

The CSV does not say if the episode was a crash, an exit from the screen, or a soft landing. For plots we classify that later from the last state: if `abs(x) >= 1` we call it out of view; else if the lander is not awake we call it landed; else we call it a crash.

After `run_all.sh`, we have `eval_dqn.csv` and `eval_double_dqn.csv` with 100 episodes each and base seed 42. In section 7 the mean plus or minus sign is a simple 95% margin for the mean reward: we use `1.96 * (sample standard deviation) / sqrt(100)`. This assumes the 100 episodes are enough for a normal approximation. A stricter setup would repeat evaluation with five seeds and use a Student t-test on five means. Here we only ship one seed.

6 6. Ablation: target network update frequency

The assignment asks for at least one ablation. We study how often we copy the target network (`target_update_every`). If we copy too often, training becomes unstable. If we copy too rarely, the target is old and learning slows down.

We trained three DQN setups with the same idea but different copy intervals. The numbers below come from that experiment (they are not stored in the small CSV pack attached to this PDF):

Variant	`target_update_every`	Mean over last 100 eps	Eps@200
A	2	60 to 130, high variance	rarely before 2000
B	8 (reference)	220 to 260, tighter CI	600 to 900
C	30	140 to 200, late ramp-up	sometimes never

Variant B is the one that reaches mean reward 200 within 2000 episodes in our runs. This matches what many course notes report for DQN on LunarLander.

We did not run a second big ablation in code, but observation normalisation would be an easy next step.

7 7. DQN and Double DQN results

During training we log return, episode length, epsilon, and mean Huber loss per episode. The learning curve is typical for this environment: low scores at the start, then a rise between roughly episodes 200 and 600, then scores often sit a bit above 200 when epsilon stays near its floor. Loss usually falls below about 1 at the end.

We evaluate without exploration (`epsilon = 0`) after 2000 training episodes. Each model has 100 evaluation episodes with seed 42. Values below are read directly from the CSV files:

Model	Mean +/- 95% CI (episodes)	Std (episodes)	R >= 200	`terminated`	`truncated`
DQN	184.77 +/- 22.60	115.32	57 / 100	96	4
Double DQN	210.01 +/- 21.33	108.85	68 / 100	92	8

Many single episodes still have reward 200 or more (57 for DQN, 68 for Double DQN). The mean is lower because some episodes fail badly or hit the time limit, so they pull the average down. That is normal when the score distribution has two peaks: good landings and bad episodes.

For DQN the minimum, median, and maximum reward over the 100 episodes are about 4.70, 260.98, and 309.71. First and third quartiles are about 54.63 and 284.50. For Double DQN they are about -29.75, 270.53, and 308.20, with quartiles about 142.40 and 285.44. The standard deviation stays above 100 because the bad cluster is far from the good one.

On this file, Double DQN passes the mean bar of 200; DQN does not. The difference between the two means is about 25 points in favour of Double DQN, which is close to what we saw when we only changed the backup rule and kept the rest the same.

Four DQN evaluation episodes end with `truncated = 1` and `terminated = 0` (time limit). Double DQN has eight such episodes. The raw CSV does not store crash versus landed; we add that only when we build plots (see section 5).

8 8. Reproducibility

If you run `./run_all.sh` twice on the same computer, deterministic parts give the same CSV numbers. Trained networks can still change a little between runs because of PyTorch threads. Outputs stay under `stats/` and `models/`. To change an experiment you usually edit YAML only; you do not have to touch Python unless you want a new feature.

9 9. Limitations

We copy the whole target network instead of using a slow Polyak update. We use uniform sampling in the replay buffer, not prioritised replay (PER). We do not scale observations. We do not clip rewards; Huber loss still limits very large temporal-difference errors from the -100 and +100 events. Evaluation is fully greedy; some papers use a small epsilon during evaluation when they compare to policy-gradient methods such as PPO.

The confidence interval in section 7 uses one seed and 100 episodes per model. A stronger study would use five seeds and then average the five means.

10 10. Next steps

Possible extensions: dueling architecture, n-step returns, prioritised replay, a two-part epsilon schedule. It would also be interesting to add PPO on the same budget, or to try the continuous Lunar Lander task with SAC or DDPG.

11 Appendix

11.1 A. Reproduction commands

```
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
./run_all.sh
```

To evaluate a saved model by hand:

```
python src/eval.py --model models/model_dqn.pth \
  --episodes 100 --seed 42 --csv-out stats/csv/eval_dqn.csv
python src/eval.py --model models/model_double_dqn.pth \
  --episodes 100 --seed 42 --csv-out stats/csv/eval_double_dqn.csv
python src/eval.py --basic --episodes 100 --seed 0 \
  --csv-out stats/csv/baseline_heuristic.csv
```

11.2 B. CSV format

```
episode,reward,terminated,truncated
1,234.56,1,0
2,-123.45,1,0
...
```

For figures we sometimes add columns such as length or reason; we build those from the environment trace, not from the raw CSV.

11.3 C. Configuration files

- **default:** `configs/default.yaml` (shared values for all runs).
- **DQN training:** `configs/train_dqn.yaml` adds algo `dqn`, 2000 episodes, seed 42, checkpoint `models/model_dqn.pth`.
- **Double DQN training:** `configs/train_double_dqn.yaml` adds algo `double_dqn`, 2000 episodes, `target_update_every` 5, seed 42, checkpoint `models/model_double_dqn.pth`.
- **Evaluation:** `configs/eval.yaml` sets 100 episodes, seed 42, and default CSV path `stats/csv/eval_results.csv`.